Projet 2025–2026 EcoBudget

Document Technique de Synthèse — Migration Kotlin Multiplatform

1. Architecture cible

EcoBudget
- **shared/commonMain** : modèles, repository de test, logique métier,état réactif, ressources textuelles et interface Compose.
- **shared/iosMain** : point d'entrée iOS.
- **androidApp** : hôte Android.
- **iosApp** : hôte SwiftUI.

Le principe retenu est de placer dans **commonMain** tout ce qui ne dépend pas du système d'exploitation : modèles, règles de calcul, état de l'application, dépôt en mémoire, composants Compose et écran principal.

Les points d'entrée restent spécifiques aux plateformes :

• **Android**: MainActivity.kt

• **iOS** : MainViewController.kt et l'hôte SwiftUI.

2. Mise en place de l'infrastructure partagée

Module **shared**

Le module partagé est configuré comme bibliothèque Kotlin Multiplatform :

```
plugins { kotlin("multiplatform")
id("com.android.library")
id("org.jetbrains.compose")
id("org.jetbrains.kotlin.plugin.compose")
}
```

Les cibles configurées sont :

• androidTarget()

• `iosX64()`

Les cibles iOS produisent également un framework statique nommé **Shared**, ce qui permet son intégration dans l'application iOS.

2.2 Dépendances communes

Le module commonMain utilise notamment :

• Compose Runtime ;

• Compose Foundation ;

• Compose Material 3 ;

• Compose Material Icons ;

• kotlinx-coroutines-core ;

• kotlinx-datetime.

Ces bibliothèques sont adaptées à l'utilisation multiplateforme.

2.3 Liaison avec Android

androidApp dépend directement de :

```
implementation(project(":shared"))
```

L'application Android conserve donc uniquement son rôle d'hôte natif tandis que l'interface et la logique EcoBudget résident dans **shared**.

3. Migration de la couche Domaine

Category.kt

Problème rencontré

Le modèle utilisait initialement une référence vers **com.example.R**, ce qui reproduisait le fonctionnement des ressources Android dans le domaine.

Solution

Le modèle utilise maintenant des valeurs métier simples :

```
enum class
Category( val label:
String, val emoji:
String )
```

Les catégories sont :

• Transport ;

• Alimentation ;

• Loisirs ;

• Logement.

Justification

Le domaine ne connaît plus le mécanisme de ressources Android. Il ne dépend que de types Kotlin standards. Il peut donc être compilé pour Android et iOS.

3.2 Transaction.kt

Problème rencontré

La transaction doit pouvoir être partagée sans dépendre d'une classe Android.

Solution

Le modèle est une **data class** Kotlin pure :

```
data class Transaction(
val id: String, val title:
String, val amount:
Double, val date:
Long, val category:
Category )
```

Justification

String, Double, Long, data class et l'énumération Category sont indépendants de la plateforme.

3.3 YearMonth.kt

Problème rencontré

La navigation mensuelle et la détermination du mois d'une transaction ne doivent pas dépendre de java.time ou java.util.Calendar.

Solution

La gestion temporelle est réalisée avec kotlinx-datetime :

• Clock.System.now()

• Instant

• TimeZone

• LocalDateTime

• LocalDate

Une méthode **plusMonths()** a également été ajoutée pour construire les données de démonstration sur les mois précédent et suivant.

Justification

kotlinx-datetime fournit une API commune à Kotlin Multiplatform et évite les classes Java spécifiques à Android/JVM.

4. Migration de la couche Données

TransactionRepository.kt

Problème rencontré

Le contrat du repository devait pouvoir être consommé par la logique commune sans connaître une technologie de stockage Android.

Solution

L'interface expose un `Flow` et des opérations suspendues :

```
interface TransactionRepository { fun
getTransactions(): Flow<List<Transaction>> suspend
fun addTransaction(transaction: Transaction) suspend
```

fun updateTransaction(transaction: Transaction)
suspend fun deleteTransaction(id: String) }

Justification

Flow et les fonctions `suspend` appartiennent à Kotlin Coroutines et sont utilisables dans une architecture KMP.

4.2 FakeTransactionRepository.kt

Problèmes rencontrés

Le repository de démonstration utilisait :

java.util.Calendar
java.util.UUID

Ces imports sont incompatibles avec **commonMain**.

Solution

• Calendar a été remplacé par YearMonth et kotlinx-datetime.

• La génération d'identifiant utilise une combinaison d'horodatage et de valeur aléatoire Kotlin.

• Les transactions restent stockées dans un MutableStateFlow.

Justification

La donnée de test reste entièrement en mémoire et ne dépend d'aucune API JVM. Le même repository peut donc être utilisé dans les deux plateformes.

5. Migration de la couche Présentation

EcoBudgetViewModel.kt

Problème rencontré

Un androidx.lifecycle.ViewModel ne peut pas être placé tel quel dans commonMain.

Solution

Le ViewModel métier a été transformé en classe Kotlin commune qui possède :

• un CoroutineScope ;

• des MutableStateFlow privés ;

• un StateFlow<EcoBudgetUiState> public ;

• une méthode clear() appelée à la destruction du composant racine.

La logique réactive utilise :

```
combine(...) stateIn(...)
SharingStarted.WhileSubscribed(...
)
```

Le scope partagé utilise Dispatchers.Default afin de ne pas imposer le dispatcher Android Dispatchers.Main au code commun.

Justification

La logique de gestion d'état est ainsi indépendante d'Android. Le cycle de vie de l'objet est contrôlé par App.kt, ce qui évite de dépendre de androidx.lifecycle.ViewModel.

5.2 EcoBudgetUiState

Cette classe centralise l'état nécessaire à l'écran :

• mois courant ;

• transactions du mois ;

• transactions filtrées ;

• catégories sélectionnées ;

• budget mensuel ;

• total dépensé ;

• montant restant ;

• état du dialogue ;

• transaction en cours d'édition.

Les propriétés calculées budgetUsageRatio, budgetUsagePercentage et isAllCategoriesSelected restent entièrement déterministes et multiplateformes.

6. Migration de l'interface Compose

Les fichiers suivants sont placés dans `commonMain` :

Fichier	Responsabilité	Traitement KMP
App.kt	Racine de l'application	Compose partagé + gestion du scope
EcoBudgetScreen.kt	Tableau de bord	UI partagée
MonthNavigatorBar.kt	Navigation mensuelle	UI partagée
AddTransactionDialog.kt	Ajout/modification	UI partagée
TransactionCard.kt	Affichage d'une dépense	UI partagée
Color.kt	Palette	API Compose commune
Theme.kt	Thème Material	API Compose commune

Type.kt	Typographie	API Compose commune

7. TransactionCard.kt : suppression des API Java

Problème rencontré

Le composant utilisait :

java.text.NumberFormat
java.text.SimpleDateFormat
java.util.Date java.util.Locale

Ces classes ne doivent pas apparaître dans commonMain.

Solution

Le formatage est centralisé dans CommonUtils.kt.

Le montant est formaté par :

formatFcfa(transaction.amount)

La date relative utilise :

formatRelativeDate(...)

Cette fonction s'appuie elle-même sur kotlinx-datetime.

Justification

Le formatage des montants et des dates ne dépend plus du runtime Java. L'interface est donc portable vers iOS.

8. Centralisation des ressources textuelles

Problème rencontré

Le projet Android utilisait normalement le mécanisme :

R.string.*
androidx.compose.ui.res.stringResource

Ce mécanisme n'est pas directement disponible comme tel dans le code commun.

Solution appliquée

Un catalogue partagé est placé dans : shared/src/commonMain/kotlin/com/example/Strings.kt

Il expose : object SharedStrings et une fonction commune : fun stringResource(id: Int, vararg args: Any): String

Les textes de l'application sont donc centralisés dans **commonMain.**

Exemples :

• nom de l'application ;

• boutons ;

• navigation mensuelle ;

• catégories ;

• messages d'état vide ;

• libellés des montants ;

• textes d'accessibilité.

Justification

L'interface n'accède plus aux ressources Android. Les libellés peuvent être consommés par Android et iOS à partir du module partagé.

9. Gestion des dates et heures

Le projet utilise maintenant **kotlinx-datetime**.

La conversion d'un mois vers un timestamp est centralisée :

timestampForMonth(year, month, day, hour)

La date est ensuite interprétée avec :

TimeZone.currentSystemDefault()

Cette approche évite les différences d'API entre JVM et Native.

Conclusion

La migration transforme ecobudget en une application structurée autour d'un module partagé KMP.

Certains éléments sont désormais regroupés dans shared. Tandis que les éléments spécifiques au fonctionnement de Android sont dans androidApp et ceux spécifiques au fonctionnement de iOS sont placés dans iosApp.